LeetCode 26: Remove Duplicates from Sorted Array

Problem in Simple Words

You are given a sorted array like:

$$[1, 1, 2, 2, 3, 4, 4, 5]$$

You must remove duplicates **in-place**, meaning you should modify the same list instead of creating a new one. The function should return the number of unique values.

For this input, the unique values are:

$$[1, 2, 3, 4, 5]$$

So the function should return:

$$5$$

After the function finishes, the first 5 positions of the array should contain:

$$[1, 2, 3, 4, 5]$$

The remaining positions do not matter.

Key Observation

The array is already **sorted**. That is the most important clue.

Because the array is sorted, all duplicates are next to each other:

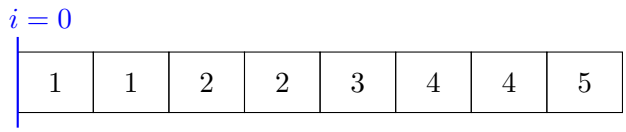
$$[1, 1, 2, 2, 3, 4, 4, 5]$$

So, when reading from left to right, a number is unique if it is greater than the last unique number we kept.

Current number	Last unique number	Keep it?
1	none	yes
1	1	no
2	1	yes
2	2	no
3	2	yes
4	3	yes
4	4	no
5	4	yes

The Main Idea

Use two mental zones inside the same array:



Input array

The variable `i` means:

The next position where we should place a unique number.

It also means:

How many unique numbers we have found so far.

At the beginning, we have found 0 unique numbers, so:

$$i = 0$$

Given Code

```
class Solution:
  def removeDuplicates(self, nums: list[int]) -> int:
    i = 0

    for num in nums:
      if i < 1 or num > nums[i - 1]:
        nums[i] = num
        i += 1

    return i
```

Line-by-Line Explanation

1. Start with `i = 0`

```
i = 0
```

This means no unique numbers have been placed yet. Think of `i` as a writing pointer. It tells us where to write the next unique number.

2. Loop through every number

```
for num in nums:
```

This reads each value from the original array one by one. The variable `num` is the value we are currently checking.

3. Decide whether this number is unique

```
if i < 1 or num > nums[i - 1]:
```

This condition has two parts.

Case A: `i < 1`

If `i = 0`, we have not stored any number yet. So the first number must always be accepted.

This means:

> If this is the first unique number, keep it.

Case B: `num > nums[i - 1]`

`nums[i - 1]` is the last unique number we placed.

Because the array is sorted, if the current number is greater than the last unique number, then it must be a new unique value.

Example:

$$\texttt{last unique} = 2, \quad \texttt{current num} = 3$$

Since:

$$3 > 2$$

we keep 3.

But if:

$$\texttt{last unique} = 4, \quad \texttt{current num} = 4$$

then:

$$4 > 4 \quad \text{is false}$$

So we skip it.

4. Place the unique number

```
nums[i] = num
```

If `num` is unique, we place it at position `i`. This gradually builds the answer at the front of the same array.

5. Move `i` forward

```
i += 1
```

After placing a unique number, the next unique number should go into the next position. So `i` moves one step right.

6. Return `i`

```
return i
```

At the end, `i` equals the number of unique elements.

Visual Dry Run

Input:

$$[1, 1, 2, 2, 3, 4, 4, 5]$$

Step	`num`	Condition	Action	Front of array
1	1	`i < 1`	keep 1	[1]
2	1	`1 > 1` false	skip	[1]
3	2	`2 > 1` true	keep 2	[1,2]
4	2	`2 > 2` false	skip	[1,2]
5	3	`3 > 2` true	keep 3	[1,2,3]
6	4	`4 > 3` true	keep 4	[1,2,3,4]
7	4	`4 > 4` false	skip	[1,2,3,4]
8	5	`5 > 4` true	keep 5	[1,2,3,4,5]

Final result:

$$i = 5$$

The first 5 values of `nums` are:

$$[1, 2, 3, 4, 5]$$

How to Think After Reading the Question

When you read this problem, think in this order:

1. What is the question really asking?

It does not ask you to create a new list. It asks you to modify the original list. So this is an **in-place array problem**.

2. What special property is given?

The array is sorted. That means duplicates are grouped together. This is the key that makes the problem easy.

3. What should I keep track of?

You need to know where to place the next unique value. That is why we create `i`.

```
i = next write position
```

4. How do I know if a value is new?

Compare the current number with the last unique number. The last unique number is at:

```
nums[i - 1]
```

If:

```
num > nums[i - 1]
```

then `num` is new.

5. What do I do when I find a new value?

Write it at index `i`, then increment `i`.

```
nums[i] = num

i += 1
```

Why This Works

At every moment, the part of the array before index `i` contains only unique numbers.

That means:

```
nums[0:i]
```

is always the clean unique part.

For example, after some steps:

```
nums[0:i] = [1,2,3]
```

If the next number is 3, skip it. If the next number is 4, write it after 3.

This is called a **two-pointer** idea:

- one pointer reads values through the loop,
- one pointer, `i`, writes unique values.

Complexity

- Time complexity: $O(n)$, because every number is checked once.
- Space complexity: $O(1)$, because no extra array is created.

Final Intuition

Think of the array as having two parts:

$$\underbrace{[1, 2, 3, 4, 5]}_{\text{unique cleaned part}} \qquad \underbrace{[\_, \_, \_]}_{\text{ignored leftover part}}$$

The variable `i` protects the clean part and always points to the next empty slot.

Whenever you see a new number, put it into that slot and move `i` forward.